Testing single_table.sql locally

CROSSFADE, info579_group_1 (Herling, Nathan; Pabari, Nidhi Nilesh). INFO 579 Project 2, Task 2. 67 tests; last run 2026-11-07 22:41:15: 67 passed, 0 failed.

How single_table.sql is checked: every query is run on the crossfade database and compared with an answer computed independently in Python from data.csv; its comment, table, columns and brief requirements are checked against the use cases; logic the sample data never reaches is tested by adding rows in rolled-back transactions; and the one statement that must fail is made to fail.

1. What the tests compare against

Source	From	How it is used
Independent answers	oracle.py + data.csv	Each query's answer is recomputed in plain Python from data.csv: no SQL, no database. MySQL must return the same columns, rows and order, to the exact decimal.
The use cases	use_cases.json ← requirements_analysis.pdf	Each query names the use cases it answers. Each use case quotes the requirements analysis word for word (build_usecases.py fails if a quote is not in it) and lists the columns it may read.
MySQL's own plan	EXPLAIN FORMAT=JSON	Proves each query reads exactly the one table its comment names, from MySQL's execution plan rather than a text search of the SQL.
The brief and the slides	info579_project02.pdf, Units 6 and 8	Requirements (a)-(e) are recognised from the course's own lists: single-row and aggregate functions (Unit 6), CASE and IF as selection control (Unit 8). IFNULL is a NULL-handling function in the slides, so it counts as neither.

2. Run them

a. What you need. A running MySQL Server 8.0.16 or later and its root password; Python 3.10 or later. The mysql command-line client and Workbench are not needed: --fresh builds the database for you.

b. Get the files. Download crossfade_single_tests.zip with the button above and extract it. It holds one folder, crossfade_single_tests, with the tests, the oracle, single_table.sql, database_implementation.sql, data.csv, use_cases.json and both guides.

```
Expand-Archive -Force $HOME\Downloads\crossfade_single_tests.zip .
cd .\crossfade_single_tests
```

c. Set up Python (once). A virtual environment with the three packages. macOS / Linux: python3 and .venv/bin/python.

```
python -m venv .venv
.venv\Scripts\python -m pip install -r requirements.txt
```

d. Run the tests. --fresh rebuilds the crossfade database from database_implementation.sql first. Without it the tests use the database as it is, after checking it holds exactly the rows in data.csv. Every write a test makes is rolled back.

```
$env:CROSSFADE_DB_PASSWORD = "your-root-password"
.venv\Scripts\python -m pytest --fresh
```

e. Read the result. One line per test, then a summary. A correctness failure prints the first row where MySQL and the oracle differ; a contract failure names the comment field, table or column. Each query's result as the tests saw it is on the single_table page (Project 2 → Files).

```
collected 67 items
test_single_table.py::test_comment_contract[S1] PASSED   [  1%]
...
========== 67 passed in 0.3s ==========
```

f. Write the queries yourself. The starter kit (Project 2 → single_table → Download starter kit) is the same folder with every comment kept and every query removed. Run the tests at any point: unwritten queries are reported by name.

3. What each group proves

Group	Tests	What it proves
contract	35/35	Each query is preceded by a purpose comment with its tag, table, use cases, business need, business question, requirements and output columns. It is one read-only SELECT with no SELECT *. MySQL's own plan (EXPLAIN) shows it reads exactly the table its comment names. The brief requirements (a)-(e) it claims are exactly the ones it uses. It reads only columns its use cases declare.
correctness	14/14	Each result equals an answer computed independently in plain Python from data.csv: same columns, same rows, same order, exact decimals. Its ORDER BY includes a unique key (or the GROUP BY columns), so no two rows can tie and come back in a chance order; and two runs agree.
behaviour	13/13	Logic the sample data never reaches: a row is added or changed inside a transaction that is rolled back, and the query must still agree with the oracle on the changed data. Each case targets one branch, boundary or tie.
expected error	1/1	The [E1] statement at the end of the file, marked EXPECTED TO FAIL, must be refused by MySQL with exactly error 1062 on uq_track_audio_checksum, and track must be unchanged afterwards.
coverage	4/4	The file as a whole: all six non-associative tables are queried; each requirement (a)-(e) is met at least twice; every use case a single table can answer is answered; tags run S1, S2, ... in order, and [E#] comes after them.

4. Files

File	Role
single_table.sql	the file under test (Task 2)
test_single_table.py	the tests: contract, correctness, behaviour cases, the expected error, file coverage
oracle.py	the independent answers: each query recomputed in plain Python from data.csv, no SQL
conftest.py	shared set-up: finds the files, connects, splits the .sql into commented statements, writes the results snapshot
use_cases.json	the use cases: their RA evidence, the columns each may read, the expected error
data.csv	the Project 1 sample data the oracle reads
database_implementation.sql	builds the database the queries run on (--fresh)
pytest.ini	pytest settings (verbose, no cache folder)
requirements.txt	pytest, PyMySQL, cryptography (MySQL 8 sign-in)
test_single_table_guide.pdf	how to run the tests locally, what they cover and what they could miss
use_case_reference.pdf	every use case: tables, columns, input, expected output, test cases
mutate_single_table.py	tests the tests: breaks single_table.sql 23 ways, one at a time, and checks every break is caught (repo only)
crossfade_single_tests.zip	the tests, the oracle, every input file and both guides, in one folder that runs as-is

5. What the tests could miss

Passing means the file agrees with its sources and refuses what they forbid. It does not mean the following:

Gap	What could slip through	What covers it instead
Independent code, not an independent reading	The oracle was written from the same use cases as the queries. If a use case was misread (who counts as payable, what 'newest' means), the query and its expected answer agree and the test passes.	Each use case quotes the requirements analysis word for word on the Use cases page; a second reader.
A small sample	8 artists, 15 tracks, 10 genres, 8 listeners, 5 models, 8 stations. Many CASE branches, boundaries and ties are reached by no row at all; they are tested only where a behaviour case adds the row.	The 13 behaviour cases, and the deliberate-mistake run below.
A fixed report date	Every 'days since' figure is measured to 2026-09-30, so results never change. A live report would use CURDATE(); that version is not tested.	Stated in the file header.

Gap	What could slip through	What covers it instead
Requirements recognised by name, not by merit	(a)-(e) are detected from the course's function lists. Whether a use is 'trivial or generic' (the brief's words) is the grader's judgement, which no test can make.	Each comment says why the function is needed for the business question.
Comments checked for shape, not truth	The tests check that every comment field is present and that each use case exists. Whether the business reasoning in the comment is sound is not testable.	The comments cite the requirements analysis section by section.
Our own column lists	'Reads only declared columns' is checked against use_cases.json, which we wrote. A wrong list there would pass a query that reads the wrong column.	build_usecases.py checks every listed column exists in the physical model.
Running in Workbench is not the same run	The tests run each statement on its own through PyMySQL. Workbench runs the file top to bottom and stops at the first error, which is [E1] by design; splitting can also differ.	A test checks [E1] comes after every query; run the file once in Workbench by hand.
One refusal	Only the duplicate-upload refusal (1062) is exercised here. Other refusals (orphans, NULLs, CHECKs) belong to the database, not to this file.	test_implementation.py checks every constraint refuses a bad write.
Speed	No query plan is judged for cost. The tables are tiny, so this matters little here.	Not required by the brief.

6. Testing the tests

mutate_single_table.py copies the file, makes one deliberate mistake, and runs every test against the copy. A mistake is caught when a test fails that passes on the real file. 23 of 23 were caught.

Mistake	Caught	First failing tests
the two HOLD reasons tested in the other order (an artist failing both gets the wrong reason)	yes (3 failed)	test_result_matches_oracle[S1], test_s1_verified_artist_without_country_is_held, test_s1_equal_rates_break_ties_on_artist_id
the artist_id tie-break left out of ORDER BY	yes (1 failed)	test_result_is_deterministic[S1]
payable artists no longer listed first	yes (3 failed)	test_result_matches_oracle[S1], test_s1_verified_artist_without_country_is_held, test_s1_equal_rates_break_ties_on_artist_id
a different report date (2026-10-01) in days under licence	yes (3 failed)	test_result_matches_oracle[S1], test_s1_verified_artist_without_country_is_held, test_s1_equal_rates_break_ties_on_artist_id
unscored and rejected recordings allowed on the shelf	yes (4 failed)	test_result_matches_oracle[S2], test_s2_unscored_new_release_stays_off_the_shelf, test_s2_same_day_releases_order_by_track_id
six recordings on the shelf instead of five	yes (4 failed)	test_result_matches_oracle[S2], test_s2_unscored_new_release_stays_off_the_shelf, test_s2_same_day_releases_order_by_track_id
run time without the zero pad (3:7 instead of 3:07)	yes (4 failed)	test_result_matches_oracle[S2], test_s2_unscored_new_release_stays_off_the_shelf, test_s2_same_day_releases_order_by_track_id
the track_id tie-break left out of ORDER BY	yes (1 failed)	test_result_is_deterministic[S2]
the composer status reversed (READY when the description is missing)	yes (2 failed)	test_result_matches_oracle[S3], test_s3_genre_without_description_is_hidden
SELECT * instead of the named columns	yes (3 failed)	test_read_only_select[S3], test_result_matches_oracle[S3], test_s3_genre_without_description_is_hidden
plan tiers listed cheapest first	yes (2 failed)	test_result_matches_oracle[S4], test_s4_tier_with_no_listeners_is_absent
average tenure not rounded	yes (2 failed)	test_result_matches_oracle[S4], test_s4_tier_with_no_listeners_is_absent
an output column renamed (listener_count)	yes (1 failed)	test_result_matches_oracle[S4]
an accuracy tie goes to the older deployment	yes (1 failed)	test_s5_accuracy_tie_prefers_the_newer_version

Mistake	Caught	First failing tests
the model_id tie-break left out (a full tie on accuracy and date picks either row)	yes (1 failed)	test_result_is_deterministic[S5]
a second table read (a subquery on station)	yes (3 failed)	test_reads_exactly_one_table[S5], test_s5_accuracy_tie_prefers_the_newer_version, test_s5_full_tie_goes_to_the_lower_model_id
the Use cases line removed from the comment	yes (2 failed)	test_comment_contract[S5], test_columns_match_use_cases[S5]
a window of exactly 0.50 counted as wide (>= instead of >)	yes (2 failed)	test_result_matches_oracle[S6], test_s6_window_of_exactly_half_is_not_wide
a column the use cases do not declare is read (listener_id)	yes (1 failed)	test_columns_match_use_cases[S6]
a release due on the report date counted as on time (< instead of <=)	yes (1 failed)	test_s7_release_due_on_the_report_date_is_late
LIMIT used but (d) not declared in Requirements	yes (1 failed)	test_requirements_match_declared[S2]
the re-upload computes a different checksum, so MySQL accepts it	yes (1 failed)	test_expected_error_is_raised[E1]
the expected-to-fail statement placed before the queries	yes (1 failed)	test_tags_are_unique_and_in_order

7. Findings

[S5] could tie (fixed Nov 7). S5 picks one row with LIMIT 1 and sorted only by accuracy and deployment date, so two versions tied on both would have made the default MySQL's choice. The strengthened order check caught it; model_id is now the last sort key (the lower one wins a full tie), the comment says so, and a behaviour case pins it. The sample data never had such a tie, so no result changed.

The tests had a blind spot (now fixed). The deliberate-mistake run showed that removing the tie-break from [S1] or [S2] went unnoticed: MySQL happened to return tied rows in primary-key order, which is the order the tie-break asks for, so the two tie-break behaviour cases still passed. The determinism test now checks that every ORDER BY includes a unique key, which catches both mistakes.